Python for Data Science — 1

ARRAY & IMAGE

คู่มือภาษาไทย: หลักการทำงาน วิธีคิด การตรวจผล ภาพประกอบ และ code examples
ครอบคลุม Exercise 00–05 ตาม subject Version 2.0





pixels → array

shape slicing RGB transpose testing

เป้าหมาย

อ่าน array ให้เป็น • ทำนาย shape ก่อนรัน • มองภาพเป็นข้อมูลตัวเลข • ออกแบบ validation และ tests • อธิบายงานของตนเองได้ใน defense

แผนที่การเรียนรู้

เรียนตามลำดับจาก data structure → array operations → image operations เพราะแต่ละช่วงใช้ mental model จากช่วงก่อนหน้า

1

Foundation

list, nested list, type hints, function, docstring

2

Array

shape, ndim, dtype, axis, indexing, slicing

3

Computation

element-wise, vectorization, broadcasting, boolean mask

4

Image

pixel, RGB channels, crop, grayscale, display

5

Transform

transpose, channel filters, invariants

6

Quality

validation, exception handling, testing, Flake8

วงจรเรียนหนึ่งหัวข้อ

1) อ่าน contract ของโจทย์ 2) วาดข้อมูลและ axes 3) ทำนาย shape/dtype/value จากตัวอย่างเล็ก 4) เขียนด้วยตนเอง 5) ตรวจ invariants และ edge cases 6) อธิบายเหตุผลโดยไม่อ่านโค้ด

ลำดับ Exercise

ex00 BMI · ex01 2D · ex02 Load · ex03 Zoom · ex04 Rotate · ex05 Filters

กฎของ Module และ Definition of Done

กฎร่วมทุก Exercise

- Python 3.10 ขึ้นไป
- explicit imports; ห้าม wildcard import
- ห้าม global variables
- ทุก function ต้องมี docstring
- โปรแกรมมี main() และ main guard
- exception ที่ไม่ถูกจัดการอาจทำให้ exercise ไม่ผ่าน
- ตรวจ style ด้วย Flake8

สิ่งที่ evaluator มองหา

- ชื่อไฟล์และ function signature ตรง subject
- output format ตรงตัวอย่าง
- valid input ให้ผลถูกต้อง
- invalid input ไม่ทำให้โปรแกรมหยุดแบบคลุมเครือ
- อธิบาย shape, dtype, axes และวิธีตรวจผลได้
- ไม่มีการใช้สิ่งที่โจทย์ห้าม

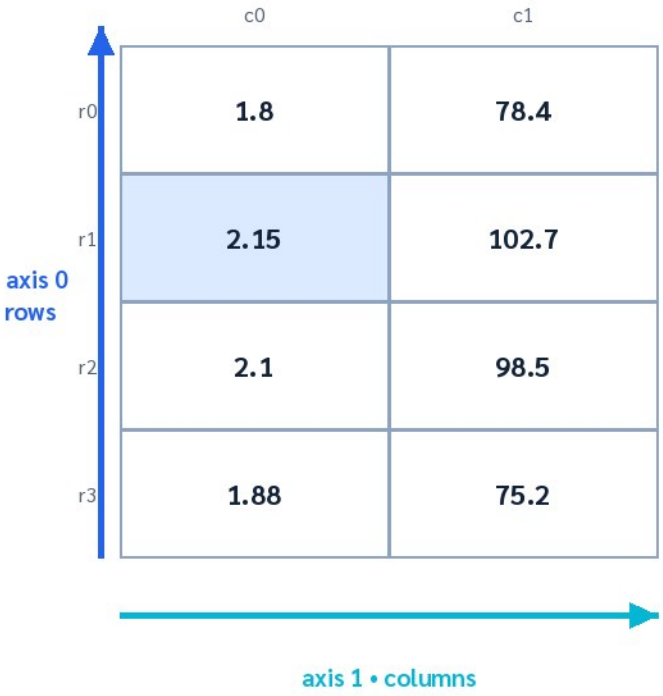
Definition of Done

1. Contract: input, output, type และ shape ตรงโจทย์
2. Correctness: tiny examples และ expected output ตรงกัน
3. Robustness: invalid types, mismatched sizes, empty/boundary cases ถูกจัดการ
4. Image invariants: dimensions, channels, dtype และ value range สมเหตุผล
5. Compliance: allowed libraries/operators และชื่อไฟล์ถูกต้อง
6. Explainability: บอกเหตุผลของ operation และ trade-off ได้

หลักคิด

“รันได้” เป็นเพียงจุดเริ่มต้น — correctness ต้องมีหลักฐานจาก contract, prediction, invariants และ tests

Array Anatomy: shape, axis, dtype



shape = (4, 2)

ตัวเลขตัวแรกคือจำนวน rows และตัวที่สองคือจำนวน columns
ถามเสมอว่าแต่ละ row และ column แทนอะไรในโลกจริง

dtype

NumPy array เก็บ element ด้วย dtype เดียวเป็นหลัก dtype กำหนด precision, memory และช่วงค่าที่คำนวณได้

Mental model ที่ต้องตอบได้

- shape บอกขนาดของทุก axis; ndim บอกจำนวน axes
- index เลือกตำแหน่งเดียว ส่วน slice เลือกช่วง
- array[row, column] — สำหรับภาพมักอ่านเป็น image[y, x, channel]
- operation อาจเปลี่ยน value, shape, dtype หรือทั้งสามอย่าง

ก่อนรัน ให้ทำนาย 4 อย่าง

1) shape 2) ndim 3) dtype 4) ตัวอย่าง value อย่างน้อยหนึ่งตำแหน่ง
เมื่อ observed result ไม่ตรง prediction ให้ตรวจ mental model ก่อนแก้โค้ด

การคำนวณแบบ Array

Element-wise
คำนวณตำแหน่งต่อ ตำแหน่ง
เช่น weights / heights²

Vectorization
สั่ง operation กับข้อมูลทั้งชุด
NumPy จัดการ loop ภายใน

Broadcasting
ทำให้ shapes บางแบบคำนวณร่วมกันได้
เทียบ dimensions จากด้านขวา

Boolean mask
ผลเปรียบเทียบเป็น True/False
ใช้ตอบคำถามหรือเลือกข้อมูล

Broadcasting: วิธีตรวจโดยไม่เดา

A เขียน shape ของข้อมูลทั้งสองฝั่ง

B เทียบแต่ละ dimension จากด้านขวา

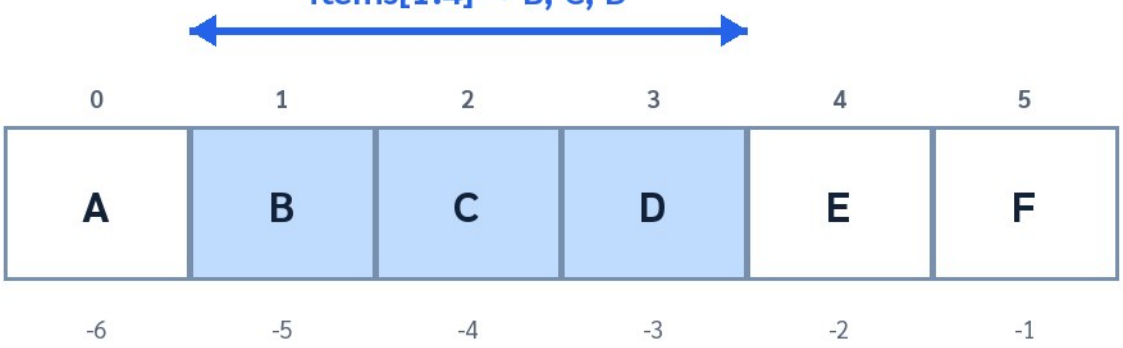
C dimension ต้องเท่ากัน หรือด้านหนึ่งมีขนาด 1

D ถ้าไม่เข้าเงื่อนไข ต้องปรับ shape หรือออกแบบใหม่

View vs Copy
NumPy slice มักเป็น view ที่แชร์ memory กับต้นฉบับ การแก้ view อาจเปลี่ยน original ด้วย
ถามก่อนแก้ข้อมูลเสมอว่า function ควรรักษา input เดิมหรือไม่ และออกแบบ test ตรวจ mutation

Slicing: เลือกช่วงอย่างมีเหตุผล

กฎหลัก: start รวม • end ไม่รวม • negative index นับจากท้าย



วิธีคิดกับ negative index

1) เขียนความยาวข้อมูล
2) แปลงค่าติดลบเป็นตำแหน่งจริง
3) ใช้กฎ start-inclusive / end-exclusive
4) นับจำนวน element ที่เหลือ

หลายแกน

2D: array[row_slice, column_slice]
Image: image[y_slice, x_slice, channel_slice]

crop เปลี่ยนพื้นที่ที่เลือก แต่ไม่ได้สร้าง pixel ใหม่แบบ resize

ข้อผิดพลาดที่พบบ่อย

- นับ end รวม ทำให้เกิด off-by-one
- สับสน negative bound กับการย้อนทิศทาง
- สลับแกน x/y ของภาพ
- ลืมตรวจ shape ของผลลัพธ์ก่อนนำไปใช้ต่อ
- แก้ slice แล้วเปลี่ยน original เพราะ slice เป็น view

Exercise 00 — Give my BMI

เป้าหมาย

รับ height และ weight หลายรายการ คำนวณ BMI และสร้างผล True/False เมื่อเทียบกับ limit

หลักการทำงาน

หนึ่ง index แทนหนึ่งคน จึงต้องรักษาการจับคู่ระหว่าง height[i] และ weigh
และเหมาะกับ element-wise operation

BMI

$$\text{weight} \div \text{height}^2$$

วิธีคิดทีละขั้น

1. เขียน input/output contract และยืนยันหน่วย: weight เป็น kg, height เป็น m
2. ตรวจชนิด จำนวนสมาชิก และค่าที่ทำให้หารไม่ได้ก่อนคำนวณ
3. คำนวณหนึ่งรายการด้วยมือเพื่อเป็น expected value
4. คำนวณทั้งชุด แล้วตรวจว่าจำนวนผลลัพธ์เท่ากับ input
5. เปรียบเทียบ BMI กับ limit เพื่อสร้าง boolean list

Error / Edge cases

- สอง list ยาวไม่เท่ากัน
- สมาชิกไม่ใช่ int/float
- height เป็นศูนย์หรือติดลบ
- empty input

หลักฐานว่าเข้าใจ

- ผลเป็น Python list
- ค่าตัวอย่างตรงการคำนวณด้วยมือ
- จำนวน boolean ตรงจำนวน BMI
- invalid input มีข้อความชัดเจน

Exercise 01 — 2D Array

เป้าหมาย

รับ 2D list แสดง shape เดิม แล้วคืนเฉพาะ rows ระหว่าง start และ end ด้วย slicing

หลักการทำงาน

2D array ต้องเป็น rectangular structure: ทุก row มีจำนวน columns เท่ากัน การ slice ในโจทย์นี้เปลี่ยนจำนวน rows แต่จำนวน columns ควรคงเดิม

วิธีคิดทีละขั้น

1. ยืนยันว่า input เป็น 2D และทุก row ยาวเท่ากัน
2. ทำนาย original shape จากจำนวน rows × columns
3. แปลงความหมายของ start/end โดยเฉพาะค่าติดลบ
4. ใช้ slicing กับแกน rows และทำนาย new shape
5. ตรวจว่า original data ไม่ถูกเปลี่ยนโดยไม่ตั้งใจ

Error / Edge cases

- ไม่ใช่ nested list
- ragged rows
- start/end ไม่ใช่ int
- slice ว่าง
- bound เกินช่วง

หลักฐานว่าเข้าใจ

- ใช้ slicing ตามข้อกำหนด
- ข้อความ shape ตรง format
- row ที่ได้ตรง expected indices
- negative index อธิบายได้

Image as Array: pixels, axes, channels

width • axis 1 • x

t • axis 0 • y



shape = (H, W, C)

H = rows / height
W = columns / width
C = color channels



หนึ่ง pixel = [R, G, B]

คำถามที่ต้องตอบก่อนแตะภาพ

- ภาพเปิดมาเป็น mode อะไร — RGB, RGBA, grayscale หรืออย่างอื่น?
- shape, ndim และ dtype เป็นอะไร? ช่วงค่าควรอยู่ที่เท่าไร?
- พิกัดกำลังเขียนเป็น (x, y) หรือ array[y, x]?
- operation จะเปลี่ยน spatial dimensions หรือจำนวน channels หรือไม่?
- ผลลัพธ์ต้องรักษา original หรืออนุญาตให้แก้ input?

Image invariant

หลัง color filter โดยทั่วไป H และ W ต้องเท่าเดิม ส่วน dtype และช่วงค่า ต้องยังแสดงผลได้อย่างถูกต้อง

Exercise 02 — Load My Image

เป้าหมาย

โหลด JPG/JPEG แสดง shape และคืน pixel content ในรูปแบบ RGB

หลักการทำงาน

เส้นทางข้อมูลคือ path → decoder → image object → RGB normalization → array สิ่งสำคัญไม่ใช่แค่เปิดได้ แต่ต้องยืนยันว่าผลมี shape (H, W, 3)

วิธีคิดทีละขั้น

1. ตรวจว่า path เป็น string และชี้ไปยังไฟล์ที่เข้าถึงได้
2. เปิดภาพภายในขอบเขตที่จัดการ error ได้
3. ตรวจ format และแปลง color mode ให้เป็น RGB
4. แปลงเป็น array แล้ว inspect shape, dtype และตัวอย่าง pixels
5. พิมพ์ข้อความตาม format และคืน array

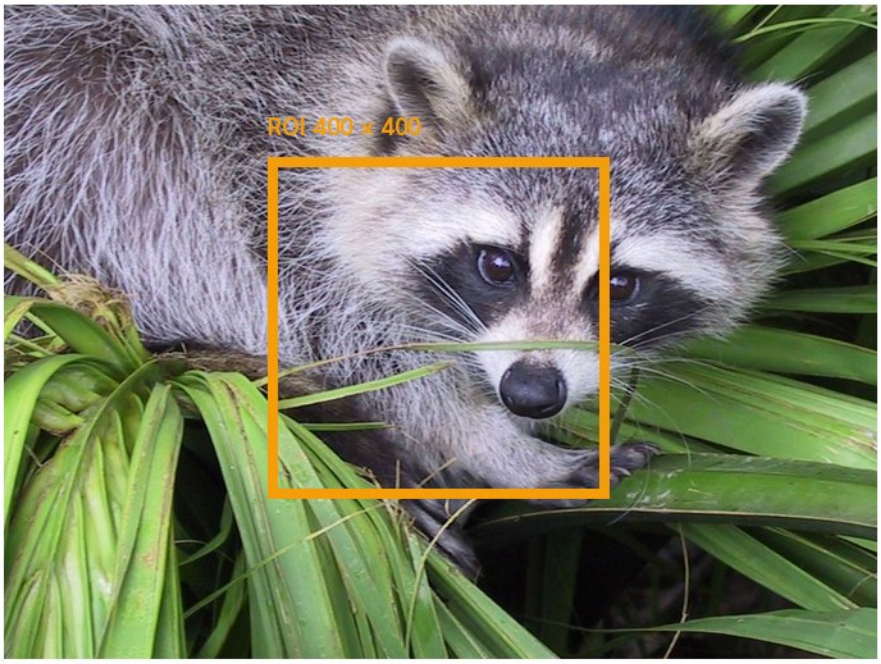
Error / Edge cases

- ไม่พบไฟล์
- permission denied
- ไฟล์เสีย
- ไฟล์ไม่ใช่ภาพ
- format/mode ไม่คาดคิด

หลักฐานว่าเข้าใจ

- รองรับ JPG และ JPEG
- channel dimension เท่ากับ 3
- shape เรียง H,W,C
- error message บอกสิ่งที่ผู้ใช้แก้ได้

Exercise 03 — Zoom on Me





grayscale crop

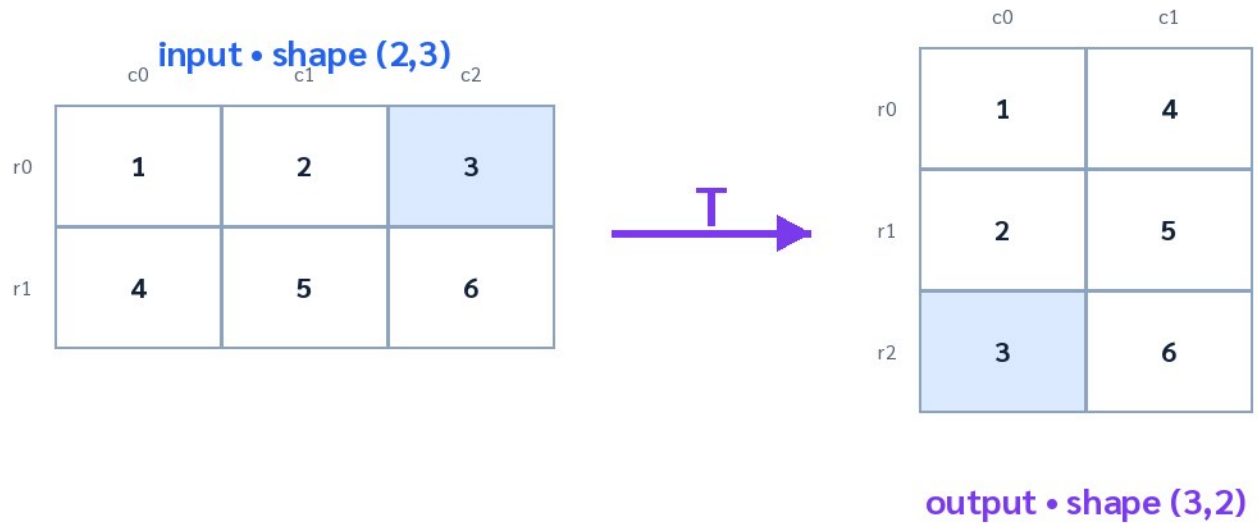
Crop ไม่ใช่ Resize

crop เลือก pixel เดิมจากช่วง y/x ส่วน resize สร้าง grid ใหม่ด้วย interpolation
โจทย์เรียกว่า zoom แต่แนวคิดหลักคือ ROI + slicing + display

วิธีคิด

- อ่าน original shape ก่อน แล้วเลือก y-range / x-range ที่ไม่เกินขอบ
- คำนวณขนาดจาก end-start ก่อน slice เพื่อป้องกัน off-by-one
- ตัดสินใจว่า grayscale ต้องเป็น (400,400) หรือ (400,400,1)
- ตรวจ new shape, sample pixels และการแสดง scale บนแกน
- จัดการกรณีภาพเล็กเกิน ROI หรือ display ไม่พร้อมใช้งาน

Exercise 04 — Transpose ด้วยตนเอง



ตำแหน่งที่ต้องจำ

output[column, row] ← input[row, column]
ตัวอย่าง: input[0,2] = 3 ย้ายไป output[2,0]
พิสูจน์ mapping ด้วย matrix เล็กก่อนทำกับภาพ 400×400

Transpose ≠ Rotate เสมอไป

Transpose สลับ axes ตามเส้นทแยงมุม ส่วนการ rotate 90° มักต้องมีการ flip ร่วมด้วย
ทิศทางสุดท้ายจึงขึ้นกับลำดับของ transpose และ flip

ข้อจำกัดของโจทย์

subject ระบุให้ทำ transpose เอง
ห้ามใช้ library สำหรับ transpose
จึงต้องอธิบาย mapping และสร้างผลลัพธ์อย่างมีเหตุผล

ตรวจผล

- shape สลับถูกต้อง
- corners ย้ายตำแหน่งถูก
- sample pixels หลายจุดตรง mapping
- output orientation ตรงที่โจทย์ต้องการ

Exercise 05 — Pimp My Image

คิดจากหนึ่ง pixel ก่อน แล้วจึง vectorize ไปทั้งภาพ



Original



Invert



Red



Green



Blue



Grey

Invariants

- H และ W เท่าเดิม
- output แสดงผลได้
- dtype/range สมเหตุผล
- input ไม่ถูกแก้โดยไม่ตั้งใจ
- ทุก function มี docstring

Operator restrictions

invert: = + - *
red: = *
green: = -
blue: =
grey: = /

ออกแบบจากผลลัพธ์ของ pixel เดียวภายใต้ข้อจำกัดนี้

คำถามก่อนทำแต่ละ filter

- pixel [R,G,B] ควรกลายเป็นค่าอะไร?
- ต้องปิดหรือผสม channel ใด?
- ลำดับ arithmetic เสี่ยง overflow/underflow หรือไม่?
- shape และจำนวน channels หลัง operation เป็นเท่าไร?

Error Handling และ Test Strategy

Validate ก่อนคำนวณ



Good error message

บอกว่าอะไรผิด + คาดหวังอะไร + ผู้ใช้แก้อะไรได้

ตัวอย่างแนวคิด: path ไม่พบ / รูปแบบภาพไม่รองรับ / input ต้องเป็น 2D rectangular list

หลีกเลี่ยง

- ปล่อย traceback จากกรณีที่คาดการณ์ได้
- except แบบกว้างจนซ่อน bug
- คืนค่ากำกวมเมื่อเกิด error
- ตรวจ error หลัง operation ที่เสี่ยงไปแล้ว

Test matrix

ประเภท	ตัวอย่าง	ตรวจอะไร
Happy path	ข้อมูลปกติ	ค่า + type + shape ตรง contract
Boundary	empty / first / last / exact limit	ไม่มี off-by-one
Invalid type	string แทน number, non-list	ข้อความชัดและไม่ crash
Invalid shape	ragged / mismatched lengths	reject ก่อนคำนวณ
Image I/O	missing / corrupt / wrong format	อธิบายสาเหตุได้
Mutation	เก็บสำเนา input ก่อนเรียก	original ไม่เปลี่ยนถ้าไม่ควร

Self Check — ข้อ 1–10

ลองตอบออกเสียงหรือเขียนในกระดาษก่อนเปิด notes ถ้าตอบได้พร้อมเหตุผล จึงถือว่าเข้าใจ ไม่ใช่เพียงจำคำศัพท์

1. shape (4,2) หมายถึงอะไร และแต่ละ axis แทนอะไร? ☐
2. dtype มีผลต่อ range, precision และ arithmetic อย่างไร? ☐
3. ข้อมูล index ใดอยู่ใน slice 1:4? ☐
4. negative index -1 และ -2 แทนตำแหน่งใด? ☐
5. element-wise operation ต่างจาก matrix multiplication อย่างไร? ☐
6. broadcasting ตรวจ compatibility จากด้านใด? ☐
7. ก่อนคำนวณ BMI ต้อง validate อะไรบ้าง? ☐
8. ragged list คืออะไรและทำไมไม่ใช่ 2D table ปกติ? ☐
9. shape (257,450,3) ของภาพแปลว่าอะไร? ☐
10. เหตุใด image array จึงใช้ y ก่อน x? ☐

Self Check — ข้อ 11–20

ลองตอบออกเสียงหรือเขียนในกระดาษก่อนเปิด notes ถ้าตอบได้พร้อมเหตุผล จึงถือว่าเข้าใจ ไม่ใช่เพียงจำคำศัพท์

- [ ] 11 crop ต่างจาก resize อย่างไร?
- [ ] 12 (H,W) ต่างจาก (H,W,1) อย่างไร?
- [ ] 13 matrix 2×3 transpose แล้ว shape ใด?
- [ ] 14 transpose เหมือน rotate 90° เสมอหรือไม่?
- [ ] 15 pixel [255,0,0] คือสีอะไร?
- [ ] 16 arithmetic กับ uint8 ต้องระวังอะไร?
- [ ] 17 เหตุใด view vs copy จึงสำคัญ?
- [ ] 18 ทำไม except แบบกว้างเกินไปจึงไม่ดี?
- [ ] 19 โปรแกรมรันได้พิสูจน์ correctness แล้วหรือยัง?
- [ ] 20 คุณอธิบาย data flow และ shape changes โดยไม่อ่านโค้ดได้ไหม?

Glossary และแผนทบทวน

shape
ขนาดของแต่ละ axis

pixel
ข้อมูล ณ จุดหนึ่งของภาพ

ndim
จำนวน axes

channel
ชั้นข้อมูลสี R/G/B

dtype
ชนิดข้อมูลของ elements

ROI
Region of Interest

axis
ทิศทางหนึ่งของ array

transpose
สลับ axes

slice
ช่วง start:end:step

invariant
คุณสมบัติที่ต้องคงจริง

vectorization
คำนวณทั้ง array

view
ข้อมูลที่แชร์ memory

broadcasting
กฎทำให้ shapes บางแบบทำงานร่วมกัน

copy
ข้อมูลแยก memory

boolean mask
array ของ True/False

validation
ตรวจ input ตาม contract

Spaced review

Day 0: เรียนและทำ tiny example • Day 1: ตอบ Self Check โดยไม่ดู notes • Day 3: อธิบาย exercise ให้คนอื่นฟัง • Day 7: ทำ edge cases และ defense rehearsal

Next action

เริ่มจาก ex00: เขียน contract และ invalid cases ลงกระดาษก่อนเปิด editor จากนั้นทำนายผลของตัวอย่างใน subject ด้วยมือ

Code Example — Array Inspection & Vectorization

ตัวอย่างนี้ใช้ข้อมูลอุณหภูมิสมมติ ไม่ใช้ชื่อ function หรือข้อมูลจาก subject จุดประสงค์คือเห็นการสร้าง array, ตรวจ metadata และคำนวณทั้งชุด

vectorized_readings.py — illustration

```python
import numpy as np

temperatures_c = np.array(
    [18.5, 21.0, 19.25, 24.0], dtype=float
)

print(temperatures_c.shape)  # (4,)
print(temperatures_c.ndim)   # 1
print(temperatures_c.dtype)

temperatures_f = temperatures_c * 1.8 + 32
above_twenty = temperatures_c > 20

print(temperatures_f.tolist())
print(above_twenty.tolist())
```

สังเกต

- shape (4,) คือ 1D array
- scalar ถูกใช้กับทุก element
- comparison คืน boolean array
- tolist() เปลี่ยนผลกลับเป็น Python list

วิธีคิด

ก่อนรัน ให้คำนวณค่าแรกด้วยมือ:
18.5 × 1.8 + 32

จากนั้นทำนายจำนวน True ใน above_twenty แล้วจึงเปรียบเทียบผลจริง

ไม่ใช่ตัวอย่างจาก subject

ข้อมูล ชื่อตัวแปร และเป้าหมายเป็นกรณีอุณหภูมิทั่วไป เพื่อสาธิต API และ mental model เท่านั้น

Code Example — 2D Slicing & Broadcasting

ตารางยอดขายสมมติ: rows แทนสาขา และ columns แทนสามเดือน ให้ทำนาย shape ของทุกตัวแปรก่อนรัน

sales_grid.py — illustration

```
import numpy as np

sales = np.array([
    [12, 15, 14],
    [20, 18, 22],
    [16, 19, 17],
    [25, 23, 28],
])

middle_rows = sales[1:3]
last_month = sales[:, -1]
window = sales[1:, 0:2]

column_mean = sales.mean(axis=0, keepdims=True)
centered = sales - column_mean

print(middle_rows.shape)  # predict first
print(last_month.shape)
print(centered.shape)
```

อ่าน slice

sales[1:3] เลือก rows index 1 และ 2
sales[:, -1] เลือกทุก row ที่ column สุดท้าย
sales[1:, 0:2] เลือกสอง columns แรกตั้งแต่ row 1

อ่าน broadcasting

column_mean มี shape (1,3) เพราะ keepdims=True
เมื่อหักออกจาก sales shape (4,3) ค่าเฉลี่ยหนึ่ง row ถูก broadcast ไปครบสี่ rows

Code Example — Load, Inspect & Crop an Image

ใช้ชื่อไฟล์สมมติ sample_photo.jpg เพื่อแสดง data flow ตั้งแต่เปิดภาพจนถึงเลือก pixel และ ROI

image_walkthrough.py — illustration

```python
from pathlib import Path
import numpy as np
from PIL import Image

image_path = Path("sample_photo.jpg")

try:
    with Image.open(image_path) as opened:
        rgb = np.asarray(opened.convert("RGB"))
except FileNotFoundError:
    print(f"File not found: {image_path}")
except OSError as error:
    print(f"Cannot decode image: {error}")
else:
    print(rgb.shape, rgb.dtype)
    sample_pixel = rgb[10, 20]
    red_plane = rgb[:, :, 0]
    crop = rgb[40:160, 80:240]
    print(sample_pixel, red_plane.shape, crop.shape)
```

Axis mapping

rgb[y, x, channel]
rgb[10,20] เลือกหนึ่ง RGB pixel
rgb[:,:,0] เลือก Red plane
crop ใช้ y-range ก่อน x-range

Error handling

FileNotFoundError แยกจาก OSError เพื่อให้ข้อความบอกได้ว่า path ไม่มี หรือมีไฟล์แต่ decoder เปิดไม่ได้

ไม่ควรใช้ except แบบกว้างถ้าเรารู้ failure modes ที่คาดได้

Code Example — Axis Swap & Color Operations

สองตัวอย่างต่อไปนี้ใช้ grid และ image array สมมติ เพื่อให้เห็น mapping ของตำแหน่งและ channel operations

position_mapping.py — illustration

```python
def swap_grid_axes(grid: list[list[int]]) -> list[list[int]]:
    row_count = len(grid)
    column_count = len(grid[0])
    swapped = [[0] * row_count for _ in range(column_count)]

    for row in range(row_count):
        for column in range(column_count):
            swapped[column][row] = grid[row][column]

    return swapped
```

pixel_operations.py — illustration

```python
import numpy as np

pixels = np.array([[[10, 40, 90], [200, 120, 30]]],
                  dtype=np.uint8)

negative = 255 - pixels
red_only = np.zeros_like(pixels)
red_only[:, :, 0] = pixels[:, :, 0]
gray = pixels.mean(axis=2, keepdims=True).astype(np.uint8)

print(negative.shape, red_only.shape, gray.shape)
```

Mapping

ตำแหน่ง (row,column) ย้ายไป (column,row)
output shape จึงสลับจาก rows×columns เป็น columns×rows

Invariants

negative และ red_only รักษา H,W,C
gray รักษา H,W แต่ channel dimension กลายเป็น 1 เพราะ keepdims=True

Code Example — Assertions as Evidence

assert ช่วยบันทึกสิ่งที่เราคาดไว้ แต่ test ที่ดีต้องตรวจทั้ง value, type, shape, range และ mutation ตาม contract

invariant_checks.py — illustration

```python
import numpy as np

values = np.array([2.0, 4.0, 6.0])
doubled = values * 2

assert doubled.shape == values.shape
assert doubled.dtype == values.dtype
assert np.array_equal(doubled, [4.0, 8.0, 12.0])
assert np.array_equal(values, [2.0, 4.0, 6.0])

image = np.zeros((5, 7, 3), dtype=np.uint8)
region = image[1:4, 2:6]

assert region.shape == (3, 4, 3)
assert region.min() >= 0
assert region.max() <= 255

print("All checks passed")
```

สิ่งที่ test นี้พิสูจน์

- shape ของ vector คงเดิม
- dtype คงเดิม
- values ตรง expected
- original ไม่เปลี่ยน
- slice dimensions ถูก
- image range ถูกต้อง

สิ่งที่ยังไม่พิสูจน์

- invalid input handling
- empty arrays
- mismatched shapes
- file I/O failures
- output text format
- allowed-library compliance

หลักใช้ code examples

อ่านทีละบรรทัด → ทำนายผล → รัน → อธิบายความต่าง → เปลี่ยนข้อมูลหนึ่งจุดและทำนายใหม่ อย่าคัดลอกโดยไม่เข้าใจ contract และข้อจำกัดของ exercise จริง